

Applied Deep Learning Coursework 2

Introduction

This coursework aims to explore weakly-supervised semantic segmentation by reducing the reliance on costly pixel-level annotations. We propose a pipeline that leverages image-level labels to train a classification model, from which Class Activation Maps (CAMs) are extracted and refined into pseudo-labels to supervise the training of a segmentation network.

Minimum Required Project(MRP)

Question: How effective are CAM-based pseudo-labels generated from image-level annotations in training segmentation models, in comparison to fully-supervised pixel-level annotations?

We evaluate the performance of segmentation models trained on CAM-derived masks against models trained with full supervision (e.g., UNet and DeepLabV3). To further analyze the pipeline, we also perform ablation studies on components of the pseudo-label generation process, including different ResNet architectures, CRF refinement, and multi-layer CAM fusion.

Open-Ended Question(OEQ)

Question: How do the quality and the type of weak annotations affect the performance of segmentation models?

To explore annotation quality, we introduce controlled levels of noise into pseudo-labels. To study the type of weak supervision, we assess the effect of incorporating bounding box constraints into the CAM generation process. These experiments reveal the robustness and sensitivity of segmentation models to different forms of weak labels.

Setup Instructions

Dataset Preparation

First, download the Oxford-IIIT Pet dataset:

```
wget https://www.robots.ox.ac.uk/~vgg/data/pets/data/images.tar.gz
```

Extract the image files:

```
mkdir -p images && tar -xzf images.tar.gz -C images --strip-components=1
```

Download annotation files:

```
wget https://www.robots.ox.ac.uk/~vgg/data/pets/data/annotations.tar.gz
```

Extract annotation files:

```
mkdir -p annotations && tar -xzf annotations.tar.gz -C annotations --strip-components=1
```

Environment Configuration

Create a new conda environment:

```
conda create -n cw2 python=3.12 pip
```

Activate the environment:

```
conda activate cw2
```

Install PyTorch as specified:

```
pip install torch==2.5.0 torchvision --index-url https://download.pytorch.org/whl/cpu
```

Install additional dependencies:

```
pip install -r requirements.txt
```

Running the Tasks

Task 1: Baseline Models Training

Navigate to the **baseline** directory:

```
cd baseline
```

To train **unet** baseline model:

```
python train_unet.py
```

To train **deeplabv3** baseline model:

```
python train_deeplabv3.py
```

Task 2: Resnet18 Training, CAM Generating and unet Training

Navigate to the `weakly-supervised` directory and run:

```
cd weakly-supervised
python train.py
```

It takes some time, after finishing, see the output images in `output-images` directory. `IOU` and `loss` value are showed on terminal.

As for `CAMs`, they are saved in `adl-cw2/weakly-supervised/data/CAM`.

Task 3: CAM Generation and Evaluation

Step 1: Generate CAMs and Train Models

Navigate to the `resnet-ablation-and-OEQ` directory:

```
cd resnet-ablation-and-OEQ
```

Then, generate the bounding box information:

```
python data/make_bbox.py
```

Then train the models:

```
python merged_train_bbox.py
```

Note: The entire process can take more than 30 minutes as it involves training ResNet50, generating CAMs, and training UNet models with and without bounding box annotations.

To generate CAM, you can run:

```
python merged_train_bbox.py
```

Step 2: Compare Pseudo Labeling Quality

After training, compare the effects of different quality pseudo labels:

```
python compare_different_quality.py
```

Step 3: Run Ablation Studies

To view our ablation experiment results:

```
python run_cam_ablation.py
```